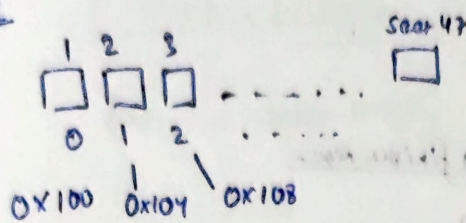


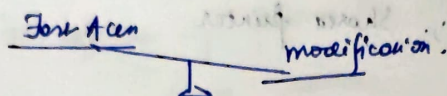
1. Array



$O(1)$

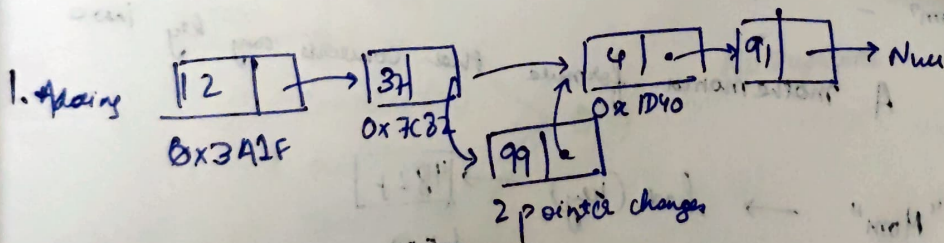
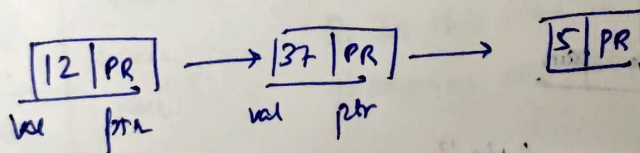
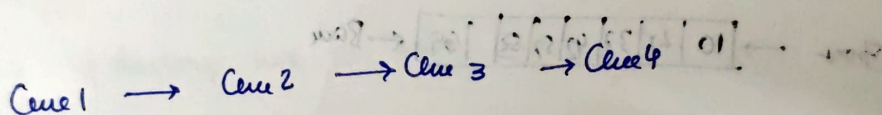
Fixed Size

New block of memory



Slow & Rigid when data is changing.

2. Linked List



2. Removing

Link is redirected.

Arrays

Adv { $O(1)$ random access
Cache-friendly

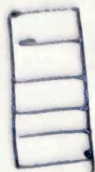
Dis { $O(n)$ insert/delete
Fixed Size (Static)

Linked List

$O(1)$ insert/delete
Dynamic size

$O(n)$ random access
Extra memory (pointers)

Stacks



LIFO

PUSH

POP

most recent → finishes next

draw-pixel()

render()

main()

Call stack
(running your code)

Queue → FIFO

Enqueue

Dequeue

E.g. Shared printer

Deque (Double Ended Queue) → Undo & redo system

Front → [10 | 25 | 37 | 42 | 52 | 52 | 69] ← Back

Hashmaps

Key - Value pairs

"Mom" - "No."

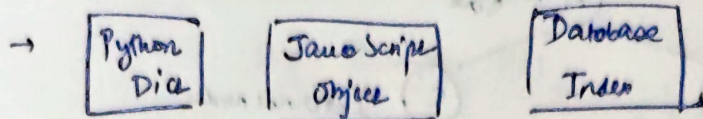
Hash function → A mathematical formula that converts any key into a number.

"Mom" → hash(key) → [482]
key function slot no.

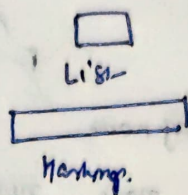
Memory slots

"Mom" → same formula → same slot → direct jump

Hashmaps in the Real world



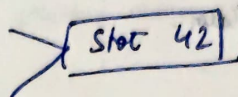
But → more memory.



→ Collision

"Cat"

"Act"



Two different keys produce the same slot no.

Now, suppose the value no longer matters

"Apple"

"Banana"

Just the keys.

→ A DS, built purely for checking membership

"Cherry" → Yes

Hashset

→ A hashmap with no values.

key	Value X
Alice	admin
Bob	user
Carol	mod

Value X
admin
user
mod

Q. Is this item in the collection or not?

Contains (x)

add (n)

remove (n)

$O(1)$

No Duplicates

→ By Design

Uses

- Tracking which users have seen a notification
- Filtering duplicates from a Dataset
- Checking if a user name is already taken.

No. all data fits into that shape

username → Alice

age → 24

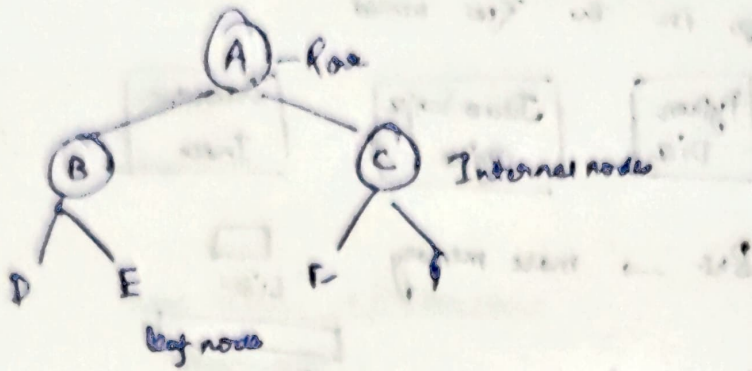
city → Tokyo

role → admin.

Direct mapping

key → some

Tree



Every HTML tag is a node, nested inside parent tags, branching into children.

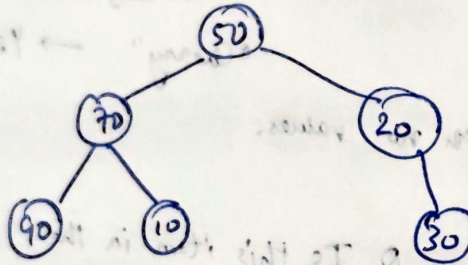
1. File Systems

2. HTML DOM

3. Common Threads

4. ML Decision Trees

A plain tree



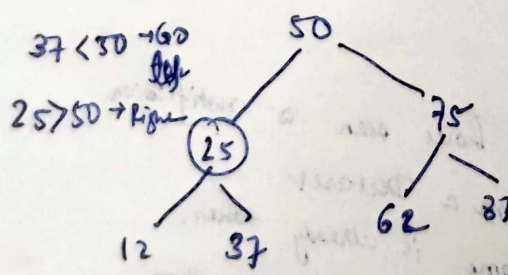
No rules about where the values go
→ Items aren't sorted

Tree — impose one simple rule on it

→ Searching faster

Binary Search Tree

The No. Guessing Game



Requires Tree

1,000,000 cases

vs BST

~20 steps

Sorted insertion:

The hidden weakness → 1 → 2 → 3 → 4

No branching, no hiding

= Linked List

$O(n) \rightarrow f(n)$

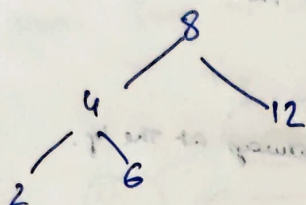
Left < Root < Right

Self-Balancing Bst



~~5 2 8 1 3~~
5 2 1 3

powerful when searching through ordered data.

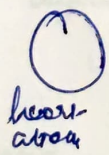


Searching is not the goal

What matters: Instant Access to the Most Important Elements

HEAP

An emergency room



heart attack

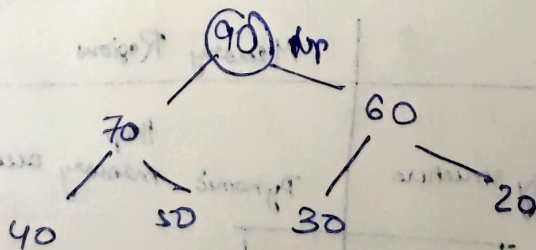


headache

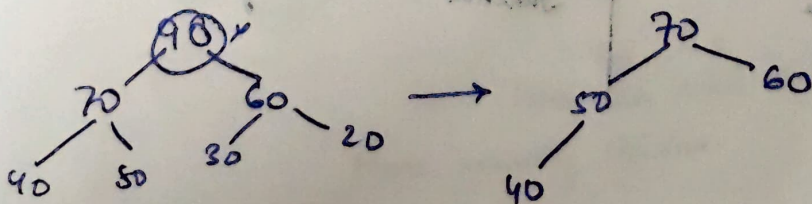
Urgency leads arrival time

highest priority → always at top.

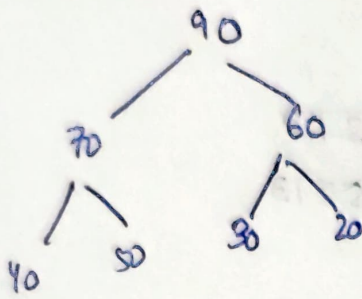
A tree that sorts itself



Self-reorganizing

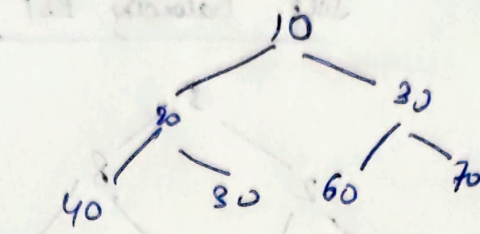


Max-heap



larger nodes to top

Min-heap

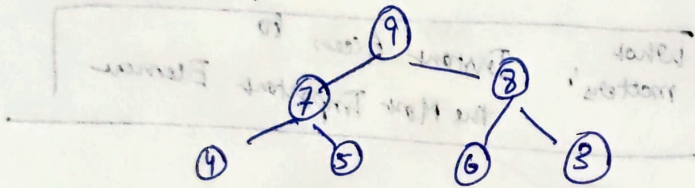


smaller nodes to top

without a heap?
→ top priority →

sort the whole list

A heap solves this.
Most important task — always at the top.



Task Scheduler

Process A
P → 9 → GPU

Process B
P → 6

Process C
P → 3

GPS Route Recalculation

Data Structures

Heap
A tree-based priority structure

Stack

LIFO data structure

Memory Regions

Heap
Dynamic memory allocation area

Stack

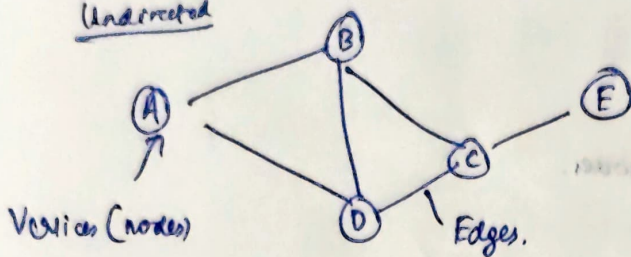
Function call & memory region

Totally unrelated

Graphs → Many
Chaotic

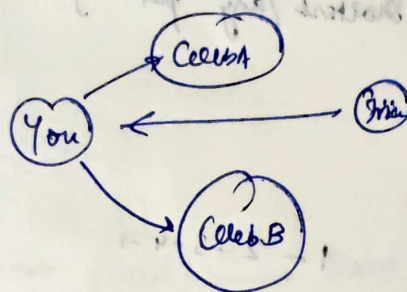
Dots and Lines

Undirected

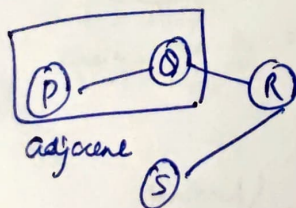


Cities are nodes. Roads are edges.

Directed Graphs

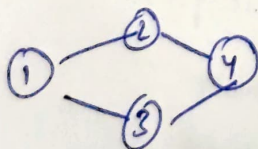


Adjacency



Storing a Graph

Way 1 - Adjacency Matrix



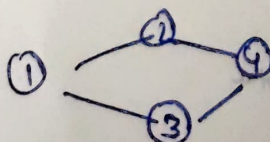
	1	2	3	4
1	0	1	1	0
2	1	0	0	1
3	1	0	0	1
4	0	1	1	0

Rows & columns } nodes

1 = edge

0 = no edge.

Way 2 → Adjacency list



1 → 2, 3

2 → 1, 4

3 → 1, 4

4 → 2, 3

Each node stores a list of its neighbours.

More memory efficient

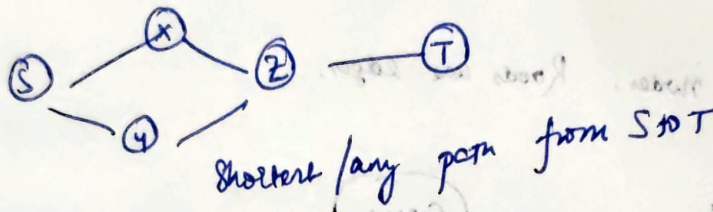
Depth first Search (DFS)

or

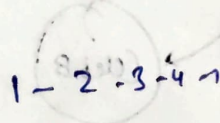
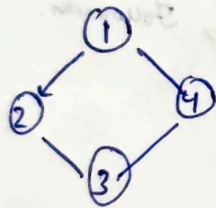
Breadth first Search (BFS) → Goes wide before going deep.

Other operations :-

i) Finding path between nodes.



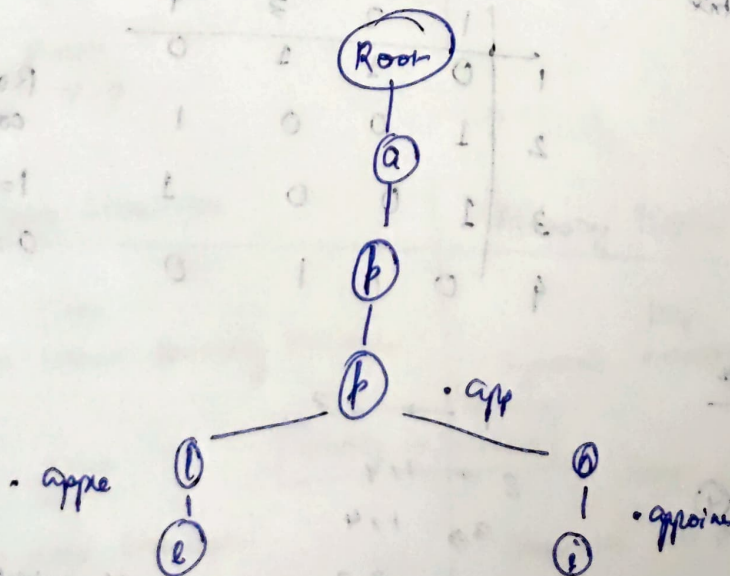
ii) Check if a cycle exists.



Tree ("Try")

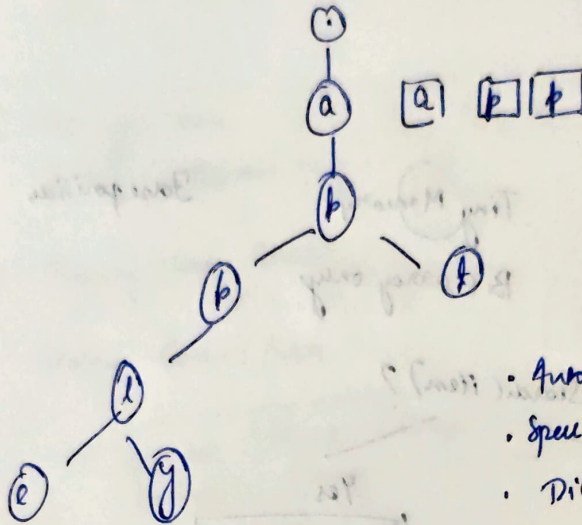
Legitimoray genes

One Letter Per Line.



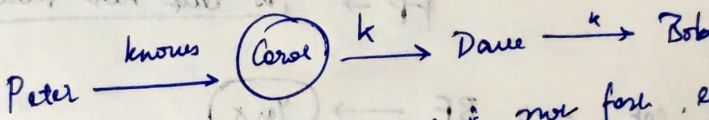
Tree: Prefix Search in Exact 4q.

Three letters $\rightarrow$ three steps taken.

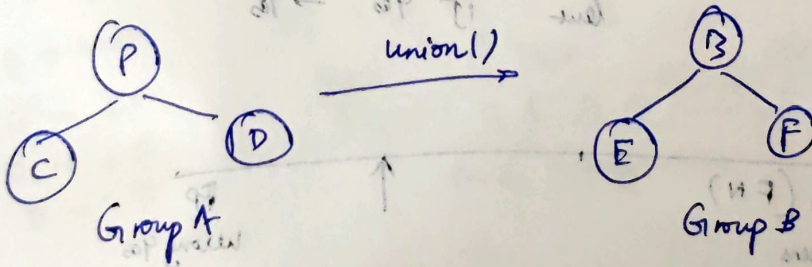


- Auto complete
- Spell checker
- Dictionary lookup

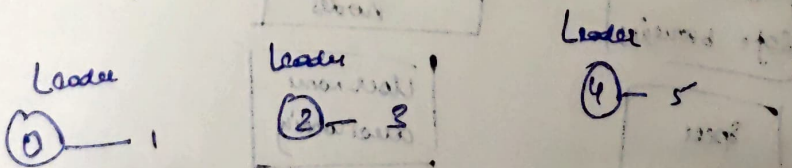
Disjoint Set (Union - Find)



Tracing every chain one-by-one is not fast enough.



Tracks group membership efficiently, even as groups keep merging & growing.



Different leader $\rightarrow$ diff. group
That there is nearly infinite

$\rightarrow$ Network Connectivity

$\rightarrow$ Image Processing

$\rightarrow$ Game World Design.

One specific case of problem:

- Tracking connected groups that keep on reducing

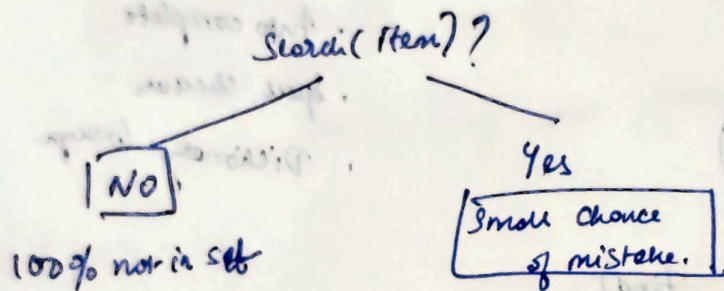
Groups keep increasing $\rightarrow$ Union - Find adaptive identity.

Phone Filter

Super Four
 $O(4)$

Tiny Memory
Bit array only

False positives



FP $\rightarrow$ Yes but not there

item: cat $\rightarrow$ BF $\rightarrow$ Yes x
not there

but if Yes $\rightarrow$ Yes

Wrong No (FN)
Answer happens

FP
Wrong Yes

Chrome
Safe browsing

Ethereum
nodes

Spice
checksum

User name
availability

"code name 99"

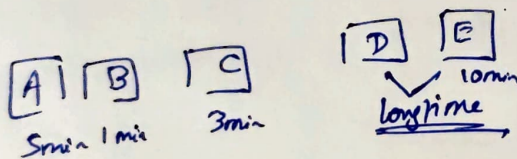
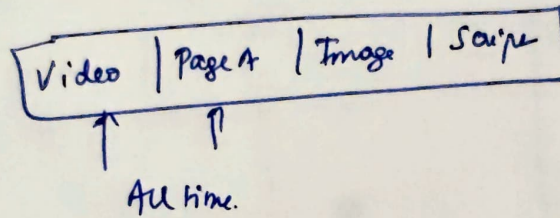
Step 1: BF → Policy taken

Step 2: Then hit database.

Smaller about when ^{to} you store
" " " " throw away.

LRU (Least Recently Used Cache)

Limited Cache / RAM

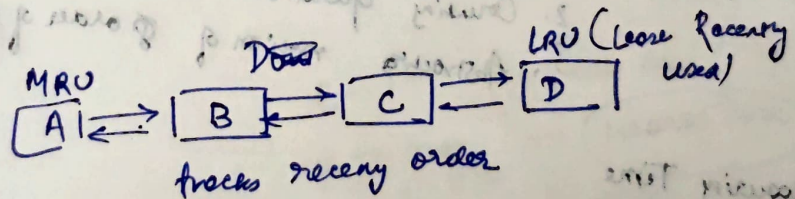


Hash Map

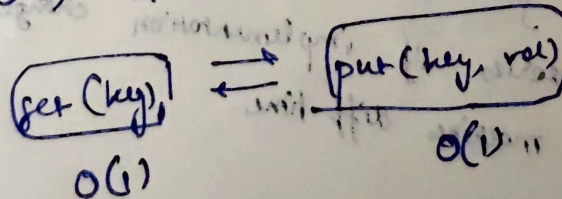
A → node A
B → " B
C → " C
D → " D

$O(1)$ lookup

+ Doubly Linked List



$O(1)$ → Get & Put



Hashmap + Doubly Linked List = Perfect cache